

Neovim Cheatsheet

Leader = **Space** · Config: kickstart + lazy.nvim · Generated 2026-05-29

Tip: **<leader>sk** searches all keymaps live, and **<leader>sh** searches help. If you forget anything on this sheet, those two find it.

Windows, Buffers & Core

Key	Action
<C-h>/<j>/<k>/<l>	Move focus to left / down / up / right window
<leader>i / <leader>I	Resize split: wider (+10) / taller (+3)
<Tab> / <S-Tab>	Next / previous buffer
<leader><leader>	Find existing buffers (Telescope)
<Esc>	Clear search highlight
<leader>a	Select all (ggVG)
<leader>cd	cd into current file's directory
yp / yP	Paste last <i>yanked</i> text (ignores deletes) below / above
<leader>x / <leader>X	Replace word under cursor, forward / reverse
<Esc><Esc>	Exit terminal mode

Search — Telescope

Key	Action
<leader>sf	Find files
<leader>sg	Grep across project (live)
<leader>sw	Grep the word under cursor
<leader>/	Fuzzy-find in current buffer
<leader>s/	Grep in open files only
<leader>s.	Recent files
<leader>sr	Resume last search
<leader>sd	Diagnostics
<leader>sh	Help tags
<leader>sk	Keymaps
<leader>ss	Telescope builtins (picker of pickers)

Inside a Telescope picker:

Key	Action
<C-n> / <C-p>	Next / previous result
<CR>	Open selection
<C-v> / <C-x> / <C-t>	Open in vsplit / split / new tab
<C-u> / <C-d>	Scroll preview up / down
<Tab>	Multi-select (mark) + move down
<C-q>	Send results to quickfix list
<C-/-> (insert) / ? (normal)	Show all picker mappings
<Esc> / <C-c>	Close

Workflow: **<leader>sg** → type a symbol → **<C-q>** to dump every match into the quickfix, then **:cdo s/old/new/g** for a project-wide edit.

Git — Neogit (**<leader>g...**)

Key	Action
<leader>gg	Open Neogit status (the main screen)
<leader>gc	Commit
<leader>gp	Pull
<leader>gP	Push
<leader>gb	Browse branches (Telescope)

Inside the Neogit status buffer:

Key	Action
<Tab>	Toggle the diff under the cursor
s / u	Stage / unstage item
S / U	Stage / unstage all
x	Discard change (careful)
c c	Commit menu → write message → <C-c><C-c> to confirm
p / P	Pull menu / Push menu
f / F	Fetch / pull
b	Branch menu · Z Stash menu · L Log menu
r / m	Rebase menu / Merge menu
<CR>	Jump to file under cursor
<C-r>	Refresh · ? help · q close

Workflow: **<leader>gg** → **Tab** to review each hunk → **s** to stage the good ones → **c c** , type message, **<C-c><C-c>** → **P** to push. **diffview.nvim** powers the diffs.

Git Hunks — Gitsigns (**<leader>h...**)

Key	Action
]c / [c	Next / previous changed hunk
<leader>hs / <leader>hr	Stage / reset hunk (works on a visual selection too)
<leader>hS / <leader>hR	Stage / reset whole buffer
<leader>hu	Undo stage hunk
<leader>hp	Preview hunk
<leader>hb	Blame current line
<leader>hd / <leader>hD	Diff vs index / vs last commit
<leader>tb	Toggle inline line-blame
<leader>tD	Toggle showing deleted lines

LSP — Code Navigation (gr...)

Key	Action
grd / grD	Goto definition / declaration
grr	Goto references
gri	Goto implementation
grt	Goto type definition
grn	Rename symbol
gra	Code action (normal & visual)
g0 / gW	Document / workspace symbols
<leader>th	Toggle inlay hints
<leader>q	Diagnostics → quickfix list
ld / ld	Previous / next diagnostic (default)
K	Hover docs (default)

Active language servers: **pyright** (Python), **lua_ls** (Lua). Manage servers with **:Mason**.

Format, Completion & Files

Key	Action
<leader>f	Format buffer (conform; also formats on save)
<leader>n	Toggle file tree (NvimTree)
<leader>N	Reveal current file in the tree

Completion — blink.cmp (insert mode):

Workflows & Power Tips

Reviewing AI-Written Code

The agent edited files; now read every change before you trust it.

Diffview (installed with Neogit) — the side-by-side review tool:

Command	Action
:DiffviewOpen	Review all uncommitted changes side-by-side
:DiffviewOpen HEAD~1	Review everything since the last commit
:DiffviewOpen main...HEAD	Review a whole branch / PR vs main
:DiffviewFileHistory %	History of the current file (what changed, when)
:DiffviewFileHistory	History of the whole repo / branch
:DiffviewClose	Close the review tab

Inside the Diffview tab:

Key	Action
<Tab> / <S-Tab>	Next / previous changed file
]c / [c	Next / previous hunk within the diff
<CR>	Open the file under the cursor in the panel
g?	Help (all mappings) · <C-w>w jump between the two panes

Review loop: **:DiffviewOpen** → **<Tab>** through every file → read each hunk → if a change is wrong, jump to it and fix or **<leader>hr** (reset hunk) → when clean, stage in Neogit and commit. Use **<leader>hd** (diff vs last commit) for a quick single-file check without leaving the buffer.

Triage tactics:

- <leader>gg** → in the status buffer, **<Tab>** expands each file's diff inline — fast first pass.
- Stage only the hunks you've verified (**s** / **<leader>hs**); leave the rest unstaged so it's obvious what you still owe a look.
- x** in Neogit (or **<leader>hr**) **discards** an AI change you don't want — surgical undo, one hunk at a time.
- <leader>hb** blames a suspicious line; **<leader>sw** on a new symbol greps the project for every other place it's used.
- Z** in Neogit stashes the working tree so you can compare "before agent" vs "after" cleanly.

Files & Project — nvim-tree

Beyond the basics on the previous page, these built-in nvim-tree keys turn it into a real file manager:

Key	Action
<C-y>	Accept completion
<C-Space>	Open menu / show docs
<C-n> / <C-p>	Next / previous item
<C-e>	Dismiss menu
<C-k>	Toggle signature help
<Tab> / <S-Tab>	Jump forward / back in snippet

Inside NvimTree:

Key	Action
<CR> / o	Open · <Tab> preview
a	Create file (end with / for a dir)
r / d / x / c / p	Rename / delete / cut / copy / paste
y / Y / gy	Copy name / relative path / absolute path
<C-v> / <C-x> / <C-t>	Open in vsplit / split / tab
H	Toggle hidden files · R refresh
E / W	Expand all / collapse all
-	Up a directory · g? help · q close

Other handy bits

- Markdown preview:** **:MarkdownPreviewToggle** (browser live preview).
- JSON files** auto-format with **jq** on save.
- TODO/FIXME** comments are highlighted; search them with **<leader>sg** for **TODD**.
- Yank** flashes the copied region so you can confirm what you grabbed.

Key	Action
<leader>N	Jump to (reveal) the current file in the tree
f / F	Live filter the tree / clear the filter
S	Search for a file in the tree
m	Mark a file · act on all marks: bd delete, bm v move, bc copy
P	Jump to parent node · < / > previous / next sibling
J / K	Jump to last / first sibling
s	Open file with system default app · . prefill a : command with its path
g?	Full mapping list

Tip: **<leader>N** is the fastest "where am I?" — it syncs the tree to whatever buffer you're in. Then **y** / **Y** / **gy** to copy the name / relative / absolute path for pasting into a prompt or terminal.

Becoming a Power User (Editor → IDE)

The three keys that compound: **.** (repeat last change), ***/#** (find word under cursor), and macros (**q**). Master these before anything else.

Multi-edit without a multi-cursor plugin — the **cgn** trick:

- *** on a word (or **/pattern<CR>**) to search for it.
- cgn** → change the **next match**, type the replacement, **<Esc>**.
- .** repeats it on the next match. **n** to skip one. Faster and safer than blind **:%s**.

Project-wide refactors:

Technique	How
LSP rename	grn on a symbol — renames everywhere safely (semantic, not text)
Grep → quickfix → edit	<leader>sg → <C-q> → :cdo s/old/new/g update
Quickfix navigation	:copen to view · :cnext / :cprev to step · :cdo {cmd} runs on every entry
Code actions	gra — imports, fixes, refactors offered by the LSP

Navigation that beats clicking:

- grd** definition → **<C-o>** to jump **back**, **<C-i>** forward. The jumplist is your browser history.

- `grr` references and `gri` implementations open in the quickfix/Telescope — `<C-q>` to keep them all.
- `g0` / `gW` — fuzzy-jump to any symbol in the file / workspace (an IDE's "Go to Symbol").
- `g;` / `g,` — hop through the **changelist** (where you last edited), independent of jumps.
- `<leader>sr` — **resume** your last Telescope search exactly where you left it.
- Set a mark with `m{letter}`; ``{letter}` jumps back from anywhere. ```` toggles between your two most recent spots.

Windows & layout (IDE-style panes):

- `:vsp` / `:sp` split; `<C-h/j/k/l>` move between panes; `<leader>i` / `<leader>I` resize.
- `<C-w>0` close every split but this one · `<C-w>=` equalize sizes · `<C-w>_` / `<C-w>|` maximize height / width.

Vim Basics Reference

A standalone refresher — independent of the config above.

Modes

Key	Enters
i / a	Insert before / after cursor
I / A	Insert at line start / end
o / O	New line below / above + insert
v / V / <C-v>	Visual char / line / block
R	Replace mode
<Esc>	Back to Normal

Motions

Key	Move
h j k l	Left / down / up / right
w / W	Next word / WORD start
b / B	Previous word / WORD start
e / ge	End of word / previous word end
0 / ^ / \$	Line start / first non-blank / line end
gg / G	Top / bottom of file
{ / }	Previous / next paragraph
%	Jump to matching bracket
f{c} / t{c}	Jump to / before next char <code>c</code> (; repeat, <code>,</code> reverse)
* / #	Next / previous occurrence of word under cursor
<C-d> / <C-u>	Half page down / up
<C-o> / <C-i>	Jump back / forward in jumplist

Operators (operator + motion/text-object)

Key	Action
d / c / y	Delete / change / yank
> / <	Indent / dedent
gu / gU / g~	Lowercase / uppercase / toggle case
=	Auto-indent
.	Repeat last change (your most-used key)

Examples: `dw` delete word · `ci"` change inside quotes · `yap` yank a paragraph · `d$` delete to line end · `>ip` indent paragraph.

Text Objects (use after an operator: `d`, `c`, `y`, `v`)

Object	Selects
iw / aw	Inner / a word
is / as	Inner / a sentence
ip / ap	Inner / a paragraph
i" i' i`	Inside quotes
i(i[i{ i<	Inside brackets (also <code>ib</code> <code>iB</code>)
it / at	Inside / around an HTML/XML tag

- Open a Telescope result in a split with `<C-v>` (vertical) or `<C-x>` (horizontal) instead of `<CR>`.

Diagnostics & quality:

- `<leader>q` dumps all diagnostics to the quickfix; `]d` / `[d` step through them in place; `<leader>sd` fuzzy-searches them.
- `<leader>th` toggles inlay hints (param names, inferred types) — closes much of the gap to an IDE.
- `<leader>f` formats on demand (also runs on save via conform); `:Mason` to add more formatters/servers.

Habit to build: when you catch yourself repeating an edit, stop and ask " . , a macro, or `cgn` ?" When you catch yourself scrolling to find something, ask " `grd` , `<leader>sg` , or a mark?" That reflex is the whole difference.

mini.ai adds: counts and next/last — e.g. `ci(` change in current parens, `cin(` change in the **next** parens, `ya2(` yank around the 2nd parens out.

Surround (mini.surround)

Key	Action
sa{motion}{char}	Add surround — e.g. <code>sa w</code> wrap word in <code>()</code>
sd{char}	Delete surround — e.g. <code>sd"</code> remove quotes
sr{old}{new}	Replace surround — e.g. <code>sr </code> turn <code>()</code> into <code>[]</code>

Marks & Jumps

Key	Action
m{a-z}	Set mark (lowercase = file-local)
`{a-z} / '{a-z}	Jump to mark / to its line
``	Jump back to position before last jump
'.	Jump to last edit

Macros

Key	Action
q{a-z}	Start recording into register
q	Stop recording
@{a-z}	Play macro
@@	Replay last macro
5@a	Play macro <code>a</code> five times

Registers & Search

Key	Action
"{r}y / "{r}p	Yank to / paste from register <code>r</code>
"0p	Paste last yank (skips deletes)
"+y / "+p	System clipboard (synced here anyway)
/ {pat} / ? {pat}	Search forward / backward
n / N	Next / previous match
:%s/old/new/g	Replace in file · add <code>c</code> to confirm each
:%s/old/new/gc	...with confirmation

Useful Commands

Command	Action
:w / :wq / :q!	Save / save+quit / quit discarding
:%!jq .	Pretty-print JSON buffer
:noh	Clear search highlight
:e {file}	Open file · <code>:sp</code> / <code>:vsp</code> split
gv	Reselect last visual selection
>> / <<	Indent / dedent line
J	Join line below onto current